1. Advanced JavaScript

1.1. Objects

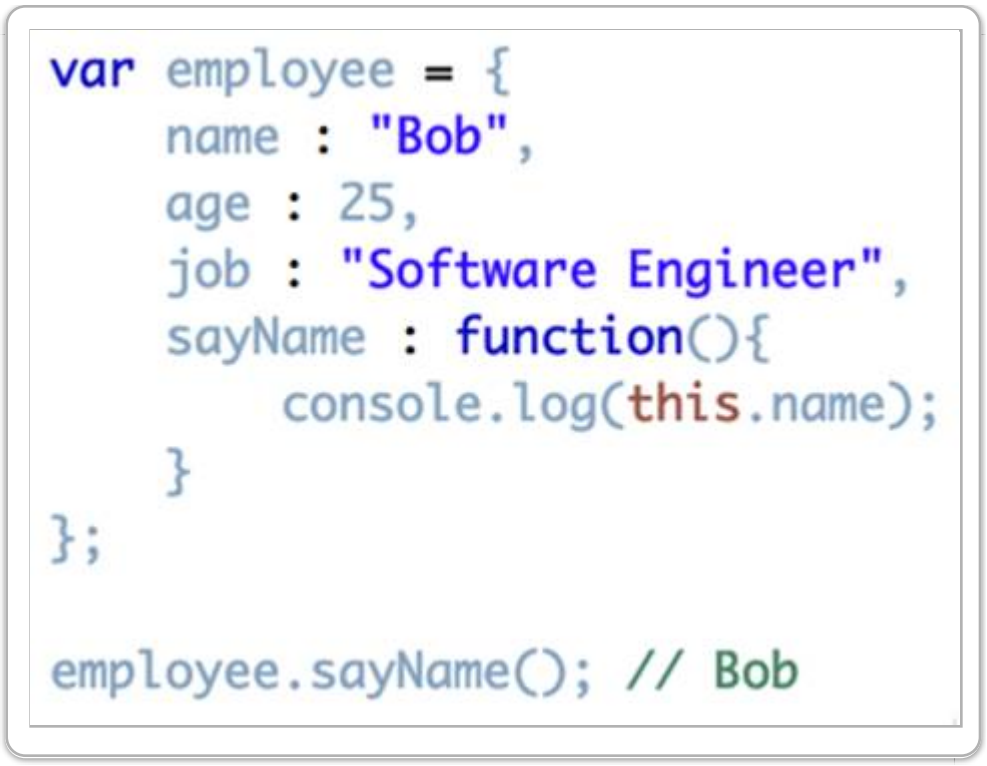
A JavaScript object is an unordered collection of properties, where each property could be a primitive value, an object, or a function. Each object is created based on a reference type. Using the *Object* constructor function, a *person* object with various properties can be initialized as shown in the following example.

```
var person = new Object();
person.name = "Bob";
person.age = 25;
person.job = "Software Engineer";
person.sayName = function(){
    console.log(this.name);
};

person.sayName(); // Bob
```

The properties defined for the object may also include functions as illustrated above. The functions may be invoked with the appropriate arguments, if any.

The object literal notation may also be used for creating the JavaScript objects as shown below.



```
var employee = {
    name : "Bob",
    age : 25,
    job : "Software Engineer",
    sayName : function(){
        console.log(this.name);
    }
};

employee.sayName(); // Bob
```

1.2. Types of Properties

The properties of JavaScript objects are classified into two categories: *data* properties and *accessor* properties. Each data property has a single location for its data value. Values are read from and written to this location. On the other hand, the accessor properties do not contain a data value. The property is manipulated using getter and/or setter function. The getter function is called when the property is read. The setter function is called with new value when property is written to.

1.3. Data Properties

The following attributes may be used when defining the data properties:

- configurable (for removing via *delete*, changing the attributes, changing to accessor)
- enumerable (whether to return in a *for-in* loop or not)
- writable (for changing the property's value)
- value (default value is *undefined*)

The following snippet defines the property, *name*, on the *person* object.

```
var person = new Object();
// same as var person =  {}

Object.defineProperty(person, "name", {
    writable: false,
    configurable: true,
    value: "Bob"
});
```

Since the *writable* attribute is set to false, changing the *name* of the person has no effect. As the *configurable* attribute is true, the property may be deleted.

```
console.log(person.name); // Bob

person.name = "Charlie";
console.log(person.name);  // Bob

delete person.name;
console.log(person.name);  // undefined
```

The *enumerable* attribute controls whether the property is included in the *for…in* loop when enumerating the properties of the object.

```
var person = {};
Object.defineProperty(person, "name", {
    enumerable : true,
    value: "Bob"
});
Object.defineProperty(person, "age", {
    value: "25"
});
Object.defineProperty(person, "job", {
    enumerable : true,
    value: "Software Engineer"
});

for (var prop in person) {
    console.log(prop, ":", person[prop]);
}
```

The output from the above example is shown below. The properties with the *enumerable* attribute set to true are only listed.

```
name : Bob
job : Software Engineer
```

1.4. Accessor Properties

The following attributes may be used when defining the accessor properties:

- configurable (for removing via *delete*, changing the attributes, changing to data property)
- enumerable (whether to return in a *for-in* loop or not)
- get (the function to call when the property is read from)
- set (the function to call when the property is written to)

Suppose a project started in the year 2010 and releases a new version every year. The object's property *year* is used to automatically set the *version* property in the following example.

```
var project = {
    _year: 2010,
    version: 1
};

Object.defineProperty(project, "year", {
    get: function(){
        return this._year;
    },
    set: function(newValue){

        if (newValue > 2010) {
            this._year = newValue;
            this.version += (newValue - 2010);
        }
    }
});
```

The initial values for the *year* and *version* will be 2010 and 1 respectively. When the user tries to set a new value for the *year*, the *version* is also modified as shown below.

```
console.log(project.year);      // 2010
console.log(project.version);   // 1

project.year = 2014;

console.log(project.year);      // 2014
console.log(project.version);   // 5
```

Multiple properties may also be defined using the *defineProperties* method as shown in the following scenario.

```
var project = {};

Object.defineProperties(project, {
    _year: {
        writable: true,
        value: 2010
    },

    version: {
        writable: true,
        value: 1
    },

    year: {
        get: function(){
            return this._year;
        },

        set: function(newValue){
            if (newValue > 2010) {
                this._year = newValue;
                this.version += (newValue - 2010);
            }
        }
    }
});
```

A sample output using the above properties is shown below.

```
console.log(project.year);      // 2010
console.log(project.version);   // 1

project.year = 2020;
console.log(project.year);   // 2020
console.log(project.version);   // 11
```

1.5. Creating Objects – Factory pattern

The Object constructor or literal notation is convenient for defining single objects. If the same functionality is required for multiple objects, all the code needs to be duplicated for each of these objects. The factory pattern encapsulates object creation through a function definition. The body of the function creates the object, assigns the properties, if any, and returns the object, as shown below.

```
function createPerson(name, age, job){
    var o = new Object();
    o.name = name;
    o.age = age;
    o.job = job;
    o.sayName = function(){
        console.log(this.name);
    };
    return o;
}
```

Using the function, different objects with the same interface can now be created as shown below.

```
var person1 = createPerson("Bob", 25, "Software Engineer");
var person2 = createPerson("Charlie", 30, "Architect");

person1.sayName();   //Bob
person2.sayName();   //Charlie

console.log(typeof person1); // object
```

1.6. Creating Objects – Constructor pattern

A drawback of the factory pattern is that the type of the object is not identified. The reference type is the object type. The constructor pattern is used for this purpose to define properties and methods for custom types.

```
function Person(name, age, job){
    this.name = name;
    this.age = age;
    this.job = job;
    this.sayName = function(){
        console.log(this.name);
    };
}
```

The constructor pattern differs from the factory pattern in the following respects. No object is explicitly created in the function. The properties and methods are assigned directly to *this* object. There is no

return statement as well.

Using the constructor pattern, objects are created using the *new* operator as shown below.

```
var person1 = new Person("Bob", 25, "Software Engineer");
var person2 = new Person("Charlie", 30, "Architect");

person1.sayName();   //Bob
person2.sayName();   //Charlie

console.log(person1 instanceof Object);  //true
console.log(person1 instanceof Person);  //true

console.log(person2 instanceof Object);  //true
console.log(person2 instanceof Person);  //true

console.log(person1.constructor == Person);  //true
console.log(person2.constructor == Person);  //true

console.log(person1.sayName);
console.log(person2.sayName);
console.log(person1.sayName == person2.sayName);  //false
```

The *sayName* properties of the two objects in the above example are functions. The function definition is not shared as each instance has its own function definition.

The constructor pattern is defining a function. If the function is called directly, the properties and functions are added to the *window* object scope as shown below.

```
// call as a function, adds to window
Person("Charlie", 30, "Architect");
window.sayName();   //Charlie
console.log(window.name); //Charlie
console.log(window.age);  // 30
console.log(job);         // Architect
```

1.7. Creating Objects - Prototype pattern

In the constructor pattern, each object has its own definition of the methods. Changing the method definition after the object is created is non-trivial, as it has to be changed for all the objects created so far. In the prototype pattern, the function has a *prototype* property. The prototype is an object containing the properties and methods for that type. These properties and methods are available to all instances of that type. All information is now assigned to the prototype as shown below.

```
function Person(){
}

Person.prototype.name = "Bob";
Person.prototype.age = 25;
Person.prototype.job = "Software Engineer";
Person.prototype.sayName = function(){
    console.log(this.name);
};
```

In the following sample, two objects are created. Both these objects have the same properties and also share the function definitions.

```
var person1 = new Person();
person1.sayName();   //Bob

var person2 = new Person();
person2.sayName();   //Bob

console.log(person1.sayName == person2.sayName);  //true
```

Once the object is created, new values can be assigned to the properties as shown below. The *name* property now is associated with the specific object, rather than the prototype.

```
var person3 = new Person();
person3.name = "Charlie";
person3.sayName();    //Charlie

console.log(person2.sayName == person3.sayName);  //true
```

If the assigned properties are deleted from the objects, the objects will still retain the values specified for the prototype as shown below.

```
delete person3.name;
person3.sayName();    //Bob
```

If a function has to be redefined, the prototype's property is used as shown below. Now, all the objects

created will use the new definition.

```
Person.prototype.sayHi = function(){
    console.log("Hi " + this.name);
};
person1.sayHi();     //Hi Bob
person2.sayHi();     //Hi Bob
person3.sayHi();     //Hi Bob
```

The object literal notation may also be used for specifying the prototype as shown the following sample.

```
function Person(){
}

Person.prototype = {
    constructor: Person,
    name : "Bob",
    age : 25,
    job : "Software Engineer",
    sayName : function () {
        console.log(this.name);
    }
};

var person1 = new Person();
person1.sayName();   //Bob
```

If an object is created before the prototype is actually defined, the functionality will not be available on the previous objects. The following example tries to invoke the function *sayName* defined on the prototype, but the object itself is created before the definition of the prototype.

```
function Person(){
}

var person1 = new Person();

 Person.prototype = {
    constructor: Person,
    name : "Bob",
    age : 25,
    job : "Software Engineer",
    sayName : function () {
        console.log(this.name);
    }
};

person1.sayName();   //error
```

The above code produces an error as shown below.

```
Object #<Person> has no method 'sayName'
```

1.8. Problems with Prototypes

With the prototype pattern, the initialization values cannot be passed as arguments to the constructor function. Also, all instances get the same property values by default. This is ideal for defining functions. Also, if the prototype defines a reference value property, this may pose problems as this reference is now shared by all the instances. In the following example, the *friends* property is a array.

```
function Person(){
}

Person.prototype = {
    constructor: Person,
    name : "Bob",
    age : 25,
    job : "Software Engineer",
    friends : ["Charlie", "Dave"],
    sayName : function () {
        console.log(this.name);
    }
};
```

In the following scenario, the *friends* of the two objects share the same array reference. If the values are modified through one object, the other objects will also see the modifications.

```
var person1 = new Person();
var person2 = new Person();

console.log(person1.friends);    //Charlie,Dave
console.log(person2.friends);    //Charlie,Dave

person1.friends.push("Ed");

console.log(person1.friends);    //Charlie,Dave,Ed
console.log(person2.friends);    //Charlie,Dave,Ed

console.log(person1.friends == person2.friends);  //true
```

1.9. Constructor/Prototype hybrid pattern

For defining custom types, the constructor and prototype patterns can be combined as follows. The constructor pattern is used for defining the instance properties. This allows arguments to be passed into the constructor. The prototype pattern is used for defining shared methods and properties. The following example makes use of the hybrid approach.

```
function Person(name, age, job){
    this.name = name;
    this.age = age;
    this.job = job;
    this.friends = ["Ed", "Fred"];
}

Person.prototype = {
    constructor: Person,
    sayName : function () {
        console.log(this.name);
    }
};
```

Now, when the objects are created, initial values are specified. The reference property, *friends*, is now unique for each object. The *sayName* function is shared among all instances.

```
var person1 = new Person("Bob", 25, "Software Engineer");
var person2 = new Person("Charlie", 30, "Architect");

person1.friends.push("Dave");

console.log(person1.friends);    //Ed,Fred,Dave
console.log(person2.friends);    //Ed,Fred

console.log(person1.friends === person2.friends);  //false

console.log(person1.sayName === person2.sayName);  //true
```

1.10. Inheritance

In object oriented programming, inheritance comes from interfaces or from implementations. In interface inheritance, only the method signatures are inherited. This is not possible in JavaScript since functions do not have signatures. In implementation inheritance, methods are inherited and in supported by JavaScript through prototype chaining.

In JavaScript functions, each constructor has a prototype object. This prototype object points back to the constructor. All instance have internal pointer to the prototype. In prototype chaining, the prototype becomes an instance of another type thus enabling the inheritance.

In the following example, properties and functions are defined on a base type, say, *SuperType*.

```
// First function
function SuperType(){
    this.property = true;
}

SuperType.prototype.getSuperValue = function(){
    return this.property;
};
```

Using the above definition, an instance is created and checked for its type as shown below.

```
var x = new SuperType();
console.log(x.getSuperValue());   //true

console.log(x instanceof Object);      //true
console.log(x instanceof SuperType);   //true

console.log(Object.prototype.isPrototypeOf(x));    //true
console.log(SuperType.prototype.isPrototypeOf(x)); //true
```

Now, to demonstrate inheritance, a second function, say, *Subtype,* is defined with its own properties. Before defining properties on its prototype, the prototype instance is first modified to be a reference to an instance of the type to be inherited from. New methods are now specified on the prototype as shown below.

```
// Second function
function SubType(){
    this.subproperty = false;
}

//inherit from SuperType
SubType.prototype = new SuperType();

// Add new methods
SubType.prototype.getSubValue = function (){
    return this.subproperty;
};
```

The *SubType* now has two methods, *getSuperValue* and *getSubValue*. The instance is now is of the *Object* type, or the *SuperType*, or the *SubType*, as shown below.

```
var y = new SubType();
console.log(y.getSuperValue());   //true
console.log(y.getSubValue());     //false

console.log(y instanceof Object);      //true
console.log(y instanceof SuperType);   //true
console.log(y instanceof SubType);     //true

console.log(Object.prototype.isPrototypeOf(y));    //true
console.log(SuperType.prototype.isPrototypeOf(y)); //true
console.log(SubType.prototype.isPrototypeOf(y));   //true
```

The inherited methods could also be overridden with a new definition. The new definition of the method is

then added to the prototype after the prototype is assigned.

```
//inherit from SuperType
SubType.prototype = new SuperType();

//new method
SubType.prototype.getSubValue = function (){
    return this.subproperty;
};

//override existing method
SubType.prototype.getSuperValue = function (){
    return false;
};
```

The following snippet shows the outcome of the two methods. The overridden definition of the method *getSuperValue* is now used for the *Subtype* object.

```
var instance = new SubType();
console.log(instance.getSubValue());   //false
console.log(instance.getSuperValue());   //false
```

1.11. Pitfalls of Prototype Chaining

If the base type contains properties with reference values, and the prototype of the derived type becomes an instance of the base type, all instances of the derived type share the same reference property. In the following example, the *colors* property is defined on the *SuperType*. Differences instances of this type

have their own instance of *colors* and hence they are not shared. However, the *SubType*'s prototype is an instance of the *SuperType.* Now, all instances of the *SubType* share the same *colors*.

```
function SuperType(){
    this.colors = ["red", "blue", "green"];
}

function SubType(){
}

//inherit from SuperType
SubType.prototype = new SuperType();
```

The above problem is illustrated below.

```
var x1 = new SubType();
x1.colors.push("black");
console.log(x1.colors);    //"red,blue,green,black"
```

```
var x2 = new SubType();
console.log(x2.colors);    //"red,blue,green,black"
```

```
var x3 = new SuperType();
console.log(x3.colors);    //"red,blue,green"
```

1.12. IIFE (Immediately Invoked Function Expressions)

IIFEs are anonymous JavaScript functions that have the following structure:

```
// Define an anonymous function
(function() {
    // code
})();   // invoke the function
```

The anonymous function is invoked right after being defined. The variables declared inside the function are not visible outside and thus avoids polluting the global namespace. Hence, all the variables declared and used inside this function have local scope.

JavaScript libraries (jQuery, etc.) use the IIFE pattern so that all the code is inside the local scope. Another common application of the IIFE is under the scenarios where the JavaScript libraries define the global functions. For example, jQuery defines the global function jQuery or $. However, the function $ is also defined globally by other libraries (e.g., prototype.js). When more than one library is being referenced in the application, the usage of $ becomes tricky.

Suppose the jQuery library is being used to define the function when the browser, as shown in the following sample, loads the document:

```
$(document).ready(function() {
    // code
});
```

In order to avoid the conflict with the $ function when other libraries are being used, the above code is wrapped in an IIFE as shown in the following sample:

```
(function($) {
    $(document).ready(function() {
        // code
    });
})(jQuery);
```

The IIFE is now defined with a function parameter. When the function is being invoked, the jQuery object is passed as the argument for the function.

Another advantage for an IIFE is the lookup time for objects in the local scope versus the global scope. When JavaScript looks up a variable, the local scope is first checked and finally the global scope. Commonly accessed global objects could be passed as arguments for an IIFE, effectively placing them in the local scope as shown in the following sample:

```
(function(doc, window, $) {
    $(doc).ready(function() {
        // code
        console.log(window.location.href);
    });
})(document, window, jQuery);
```

In the above example, when the function is immediately invoked, the global objects, *document, window,* and *jQuery* as passed as arguments for the specified parameters. Within the body of the function, the references to the parameters are now in the local scope of the function.

1.13. Module Pattern

The original purpose of the Module pattern is to provide both private and public encapsulation for classes. In the case of JavaScript, the pattern is used to include both the public/private methods and classes inside a single object. This approach reduces the function name conflicts and also restricts the variables/functions to local scope. The module pattern uses closures to encapsulate the privacy and returns a public API. Everything else is private within the closure. The module pattern is similar to IIFE. However, the pattern returns an object rather than a function.

The following example defines the function with a local variable, *counter*. The object returned by the function has two properties, *incrementCounter* and *resetCounter*. The functions defined by these two properties access the *counter* variable. The function is immediately invoked and the returned object is assigned to the variable, *testModule*.

```
var testModule = (function() {
    var counter = 0;
    return {
        incrementCounter : function() {
            return counter++;
        },
        resetCounter : function() {
            console.log("current counter value: " + counter);
            counter = 0;
        }
    };
})();
```

The properties of the object returned are shown below.

```
> testModule
  ▶ Object {incrementCounter: function, resetCounter: function}
```

The functions can now be invoked as shown below. Note that these functions manipulate the *counter* variable, which is not accessible to the user.

```
// Increment the counter
console.log(testModule.incrementCounter());
console.log(testModule.incrementCounter());

// Check the counter value and reset
// Outputs: 2
testModule.resetCounter();
```

The output of the above snippet is shown below.

```
0
1
current counter value: 2
```

Without the Module pattern, the *counter* variable would be in the global namespace, and hence any function could read and manipulate its value. As a result of this pattern, other parts of the code outside this module cannot directly read these values. The *counter* variable is effectively shielded from the global scope. The methods themselves are effectively namespaced (*modulename.function*)

The generic template for the module pattern is shown below.

```
var myNamespace = (function () {
  var myPrivateVar, myPrivateMethod;

  // A private variable
  myPrivateVar = 0;
  // A private function which logs any arguments
  myPrivateMethod = function( foo ) {
      console.log( foo );
  };

  return {
      // A public variable
      myPublicVar: "foo",

      // A public function utilizing private data
      myPublicFunction: function( bar ) {
          // Increment the private counter
          myPrivateVar++;
          // Call the private method using bar
          myPrivateMethod( bar );
      }
    };
 })();
```

1.14. Module Pattern Variations

The function can have arguments and is usually used for passing global variables as arguments. In the following example, the libraries *jQuery* and *underscore* are used. The global variables defined from these libraries are passed as arguments when the function is invoked immediately.

```
<body>
    <div class="container"></div>

    <script type="text/javascript">
        var myModule = (function(jQ, underScore) {

            function privateMethod1() {
                jQ(".container").html("Underscore min test: " +
                            privateMethod2());
            }

            function privateMethod2() {
                return underScore.min([100, 50, 20, 80]);
            }

            return {
                publicMethod : function() {
                    privateMethod1();
                }
            };
            // Invoke with jQuery and Underscore
        })(jQuery, _);

        myModule.publicMethod();
    </script>
</body>
```

The web page now renders the output as shown below.



1.15. Revealing Module Pattern

A variation of the module pattern, the revealing module pattern defines all functions and variable in the private scope. When the function is immediately invoked, the anonymous object returned by the function provides pointers so as to reveal the private functions as public as shown in the following example.

```
var myRevealingModule = (function () {

    var privateCounter = 0;
    function publicFunction() {
        privateCounter = 0;
    }
    function privateFunction() {
        privateCounter++;
    }
    function publicIncrement() {
        privateFunction();
    }
    function publicGetCount(){
      return privateCounter;
    }

// Reveal public pointers to
// private functions and/or properties

    return {
        start: publicFunction,
        increment: publicIncrement,
        count: publicGetCount
    };
})();
```

The properties of the object returned are shown below.



The functions can now be invoked as shown below.

```
  // Usage
myRevealingModule.start();
myRevealingModule.increment();
myRevealingModule.increment();
console.log(myRevealingModule.count());
```

1.16. Singleton Pattern

The singleton pattern restricts instantiation of a class to a single object. It serves as a shared resource namespace and isolates implementation code from the global namespace. The pattern provides a single point of access for the defined functions. The template for the pattern is shown below.

```
var mySingleton = (function() {

    // Instance stores a reference to the Singleton
    var instance;

    function init() {
        // Code - Private methods and variables

        return {
            // Code - Public methods and variables
            number : Math.random()
        };
    };

    return {
        // Get the Singleton instance
        getInstance : function() {
            if (!instance) {
                instance = init();
            }
            return instance;
        }
    };
})();
```

The returned object has a single property, *getInstance*, as shown below.

```
> mySingleton
  ▶ Object {getInstance: function}
```

The following snippet shows that the instances returned through multiple invocations are indeed the same.



1.17. JSON (JavaScript Object Notation)

JSON is a data format used for data exchange. The format is easy for humans to comprehend and for the machines to parse and generate. JSON is language independent and is used by several programming languages. The JSON data consists of

- simple values—such as strings, numbers, Booleans, and null
- objects—ordered key-value pairs, where the value can be a primitive type are a complex type
- arrays—ordered list of values, accessed via index. The values could be simple values, objects, or arrays.

The following example defines a JSON object.

```
var person1 = {
    "name" : "Bob",
    "friends" : ["Dave", "Ed"],
    age : 25,
    job : "Software Engineer"
};
```

The object can be converted into text using the *stringify* method as shown below.

```
var jsonText = JSON.stringify(person1);
console.log(jsonText);
```

The text representation of the data is shown below.

```
{"name":"Bob","friends":["Dave","Ed"],"age":25,"job":"Software Engineer"}
```

If the data is available in the text format, it can be converted back to an object using the *parse* method as shown below. Once the text is parsed, the data is accessed through the properties.

```
var person2 = JSON.parse(jsonText);
console.log(person2.job);
console.log(person2.friends);
```

The output of the above test is shown below.

```
Software Engineer
["Dave", "Ed"]
```

For filtering only a subset of properties in the text conversion, the second argument of *stringify* can be used to specify the array of property names as shown below.

```
var person1 = {
    name : "Bob",
    friends : ["Dave", "Ed"],
    age : 25,
    job : "Software Engineer"
};

var jsonText1 =
    JSON.stringify(person1, ["name", "job"]);
console.log(jsonText1);
```

The output now includes only the specified properties and its associated data.

```
{"name":"Bob","job":"Software Engineer"}
```

For outputting the text in an indented fashion, the third argument of *stringify* may be used to specify the number of spaces to be used for each level.

```
var jsonText2 = JSON.stringify(person1, null, 2);
console.log(jsonText2);
```

The indented output is shown below.

```
{
  "name": "Bob",
  "friends": [
    "Dave",
    "Ed"
  ],
  "age": 25,
  "job": "Software Engineer"
}
```

A replacer function may also be used during the conversion process. The function will be invoked for each property and receives the key and the value. The value returned by the function is used in the text output. If the function returns *undefined*, the respective property is omitted from the output.

```
var person1 = {
     name : "Bob",
     friends : ["Dave", "Ed"],
     age : 25,
     job : "Software Engineer"
 };

 var jsonText =
     JSON.stringify(person1, function(key, value){
         switch(key){
             case "friends":
                 return value.join(",")
             case "age":
                 return undefined;
             default:
                 return value;
         }
     });
 console.log(jsonText);
```

The output of the above sample is shown below.

```
{"name":"Bob","friends":"Dave,Ed","job":"Software Engineer"}
```

If the *toJSON* property is defined as a function on the object, the value returned by the function is used when *stringify* is invoked.

```
<script type="text/javascript">
   var person1 = {
        name : "Bob",
        friends : ["Dave", "Ed"],
        age : 25,
        job : "Software Engineer",
        toJSON: function()
            {
              return this.name + " - " + this.job;
            }
    };


    var jsonText = JSON.stringify(person1);
    console.log(jsonText);
```

The output of the above code is shown below.

```
"Bob - Software Engineer"
```

The replacer function may also be used when parsing the text back to the object notation. In the following example, the *hireDate* property is a *Date* object.

```
var person1 = {
    "name" : "Bob",
    "friends" : ["Dave", "Ed"],
    age : 25,
    job : "Software Engineer",
    hireDate: new Date(2012, 4, 15)
};

var jsonText = JSON.stringify(person1, null, 2);
console.log(jsonText);
```

In the text representation, the *string* representation of the *Date* object is used for the *hireDate*.

```
{
  "name": "Bob",
  "friends": [
    "Dave",
    "Ed"
  ],
  "age": 25,
  "job": "Software Engineer",
  "hireDate": "2012-05-15T04:00:00.000Z"
}
```

When the text is parsed, the *hireDate* property gets the string instead of the *Date*.

```
var person2 = JSON.parse(jsonText);
console.log(person2.hireDate);
console.log(typeof person2.hireDate);
```

The output of the above code is shown below.

```
2012-05-15T04:00:00.000Z
string
```

To successfully convert the string back into the *Date* object, the replacer function is used to convert the text value to the *Date* object while parsing.

```
var person3 = JSON.parse(jsonText,
    function(key, value){
        if (key == "hireDate"){
            return new Date(value);
        } else {
            return value;
        }
    });

console.log(person3.hireDate instanceof Date);
console.log(person3.hireDate.toString());
console.log(person3.hireDate.getFullYear());
```

The *hireDate* property is now a *Date* object as illustrated below.

```
true
Tue May 15 2012 00:00:00 GMT-0400 (EDT)
2012
```

1.18. References

1. Professional JavaScript for Web Developers, Nicholas C. Zakas, 3rd edition, Wiley, 2012